

SØRBLIKKET · TEKNISK DOKUMENTASJON

Dokument 2 av 2

Dataflyt og Tekniske Detaljer

Steg-for-steg gjennomgang av pipelinen

Dette dokumentet beskriver den eksakte dataflyten gjennom systemet — fra henting av partiprogrammer til statusvisning i nettleseren. Inkluderer kode-eksempler, feilhåndtering, miljøvariabler og driftsanvisninger.

A Fullstendig dataflyt**B** AI-prompter forklart**C** Miljøvariabler**D** Feilhåndtering**E** Frontend og navigasjon**F** Driftsanvisning

A — Fullstendig dataflyt

Dataflyten fra råkilde til brukerens skjerm, steg for steg.





Tidslinje for en løftes liv

- T=0 — PARTIPROGRAM PUBLISERES**

Partiet legger ut sitt valgprogram 2025–2029 som PDF på sin nettside. Systemet kan nå hente og behandle det.
- T+1 TIME — EKSTRAKSJON**

Utvikler kjører `extract-promises.mjs`. Claude leser hele programmet og identifiserer alle løfter. De lagres i databasen med status `pending`.
- T+2 TIMER — MATCHING**

Utvikler kjører `match-promises.mjs`. Claude kobler løftet til én eller flere stortingssaker. Koblingen lagres med status `pending`.
- T+N MÅNEDER — VOTERT PÅ STORTINGET**

Stortinget behandler en sak som er matchet mot løftet. Voteringsdata blir tilgjengelig i Stortingets API.
- T+N MÅNEDER + NESTE KJØRING — STATUS OPPDATERT**

Utvikler kjører `update-status.mjs`. Systemet teller FOR/MOT-stemmer og oppdaterer status til `fulfilled`, `broken`, eller `partial`.
- ALLTID — SYNLIG PÅ NETTSIDEN**

Brukere kan se alle løfter og deres status på Parti-siden i sanntid.

B — AI-prompter forklart

Claude AI mottar nøye utformede instruksjoner for hvert av de to oppgavene. Her er logikken bak dem.

Ekstraksjonsprompt — "Finn alle løfter"

Prompten er bevisst aggressiv — den ber Claude finne *absolutt alle* løfter, ikke bare de mest åpenbare. Den spesifiserer eksplisitt hvilke typer løfter som skal inkluderes:

INKLUDER DISSE LØFTETYPENE

- ✓ Tallfestede mål ("øke med 20%", "innen 2030")
- ✓ Forbudsløfter ("avvikle", "forby", "stanse")
- ✓ Handlingsplaner ("innføre", "opprette", "etablere")
- ✓ Finansieringsløfter ("bevilge", "øke støtten")
- ✓ Reform-løfter ("endre", "reformere")
- ✓ Garantier ("sikre", "bevare", "opprettholde")

EKSKLUDER DISSE

- ✗ Generelle verdier ("vi tror på")
- ✗ Analyser av situasjonen
- ✗ Historisk beskrivelse
- ✗ Vage intensjoner uten konkret handling
- ✗ Repeterte formuleringer

Returformat fra Claude (ekstraksjon)

```
// Claude returnerer en JSON-liste:  
[  
  {  
    "tekst": "Ap vil innføre gratis SF0 for alle elever på 1.-4. trinn.",  
    "kategori": "Utdanning"  
  },  
  {  
    "tekst": "Vi vil redusere norske klimagassutslipp med 55% innen 2030.",  
    "kategori": "Klima"  
  }  
]
```

```
}  
]
```

Matchingprompt — "Koble løfter til saker"

Claude mottar to lister: alle løfter fra ett parti, og alle stortingssaker. Prompten gir Claude politisk kontekst slik at den kan matche riktig:

Kontekstelementet	Hva Claude får vite	Hvorfor det hjelper
Partirolle	Om partiet er i regjering, støtteparti eller opposisjon	Opposisjonspartier fremmer representantforslag, regjeringen fremmer proposisjoner
Dokumentgruppe	Om saken er representantforslag, proposisjon, melding, innstilling osv.	Representantforslag fra opposisjonsparti er mer relevant for opposisjonspartiets løfter
Emner	Stortingets eget emne-felt for hver sak	Forenkler filtrering og gir raskere match
Regjeringens plan	Fulltekst av "Regjeringens plan for Norge 2025–29"	Hjelper Claude å matche Ap-løfter mot regjeringens egne saker

Returformat fra Claude (matching)

```
// Claude returnerer koblinger med begrunnelse:  
{  
  "matches": [  
    {  
      "lofte_id": "uuid-abc-123",  
      "sak_id": "2025-123456",  
      "notat": "Saken gjelder utvidelse av SF0-ordningen,  
               direkte relevant for løftet om gratis SF0."  
    }  
  ]  
}
```

Viktig: Claude er instruert om å *ikke* sette en øvre grense for antall matcher. Systemet er designet for å finne alle relevante koblinger, ikke bare de mest åpenbare. Antall tokens er satt til 8 192 for

matchingprompten og 32 000 for ekstraksjonsprompten.

C — Miljøvariabler

Alle hemmeligheter og tilkoblingspunkter konfigureres via miljøvariabler — aldri hardkodet i kildekoden.

Filstruktur

```
agderbenken/
├─ my-app/
│   └─ .env.local           ← Alle nøkler (ikke i Git)
└─ .env.local              ← Alternativ plassering for script
```

Alle variabler

Variabel	Hvem bruker den	Tilgangsnivå	Beskrivelse
VITE_SUPABASE_URL	Frontend + Backend	Offentlig (VITE_)	URL til Supabase-prosjektet
VITE_SUPABASE_ANON_KEY	Frontend (nettleser)	Offentlig (VITE_)	Kun lesing med RLS-begrensninger
SUPABASE_SERVICE_KEY	Backend-skript	Hemmelig	Full skrive tilgang — aldri til frontend
ANTHROPIC_API_KEY	Backend-skript	Hemmelig	Tilgang til Claude AI API
LOVDATA_API_KEY	Backend-skript	Valgfri	Lovdata-søk. Hopper over om mangler.

Vite-prefikset forklart: Variabler med `VITE_`-prefiks er tilgjengelige i nettleseren via `import.meta.env.VITE_xxx`. Disse kan trygt sees av brukere. Variabler *uten* prefiks er kun tilgjengelig i

Node.js-prosessen og når aldri nettleseren.

Eksempel .env.local

```
# Supabase – finn verdiene i Supabase Dashboard → Project Settings → API
VITE_SUPABASE_URL=https://abcdefghij.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_SERVICE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

# Anthropic – finn på console.anthropic.com
ANTHROPIC_API_KEY=sk-ant-api03-...

# Lovdata – valgfri, deaktiveres automatisk om tom
LOVDATA_API_KEY=
```

Konfigurasjonsoppslag i skriptene

Skriptene laster `.env.local` manuelt ved hjelp av en innebygd parser (ingen dotenv-pakke) og leter i to steder:

```
// Prøver begge plasseringer – første treff vinner
loadEnv(new URL("../my-app/.env.local", import.meta.url).pathname)
loadEnv(new URL("../.env.local", import.meta.url).pathname)
```

D — Feilhåndtering

Systemet håndterer en rekke kjente feilsituasjoner. Her er de viktigste, med løsninger.

Feil	Årsak	Håndtering
HTTP 429 Too Many Requests	Stortingets API har rate-limiting per minutt	Ekspontentiell backoff: 4s → 8s → 12s → 16s. Logges til konsoll.
HTTP 429 fra PDF-nedlasting	Partinettstedene begrenser nedlastinger	Retry med alternativ URL. Rødt har reservelenke i <code>retry-sp-r.mjs</code> .
400 Credit balance too low	Anthropic API-kreditter er tomme	Kan ikke håndteres i kode. Brukeren må lade opp på console.anthropic.com.
Streaming timeout	Claude bruker mer enn 10 min på store svar	Streaming er alltid aktivert for <code>max_tokens > 4096</code> . Bruker <code>stream.finalMessage()</code> .
Supabase <code>null</code> - klient	Miljøvariabler mangler i frontend	Supabase-klienten returnerer <code>null</code> . Parti-siden viser en forklaring.
Offline / nettverksbrudd	Bruker mister internettforbindelsen	OfflineBanner-komponenten viser rød melding. Grønn "tilkoblet igjen" i 3 sek ved gjenoppkobling.
Ukjent URL	Bruker navigerer til ikke-eksisterende side	React Router <code><Navigate to="/" replace /></code> sender til forsiden.
Lovdata API mangler nøkkel	<code>LOVDATA_API_KEY</code> ikke satt	Advarselsmelding til konsoll. Lovdata-søk hoppes over — resten kjører normalt.

retry-skriptene

To spesialiserte retry-skript håndterer partier som mislykkes i hoveddriften:

RETRY-SP-R.MJS

RETRY-MISSING.MJS

Kjøres manuelt etter feil på Sp og R (Rødt).
Bruker alternative PDF-URLer for Rødt, som har hatt ustabile primær-URLer. Gjentar kun disse to partiene.

Identifiserer partier som mangler løfter i databasen og kjører ekstraksjon kun for disse. Brukes etter delvis mislykkede pipelines.

SIGPIPE ved piping til head

Advarsel: Kjør aldri `node scripts/match-promises.mjs | head -30`. `head`-kommandoen avslutter prosessen med SIGPIPE når den har mottatt nok linjer, noe som dreper Node.js-prosessen midt i en API-spørring. Bruk heller:

```
node scripts/match-promises.mjs > /tmp/match.log 2>&1 &
```

E — Frontend og navigasjon

Temahåndtering — Brave-kompatibel mørk modus

Temaet er implementert med en kombinasjon av CSS og JavaScript for å fungere optimalt i alle nettlesere, inkludert Brave som ikke alltid sender `matchMedia` -endringshendelser korrekt.

Modus	DOM-tilstand	Mekanisme
<code>null</code> (system)	<code><html></code> uten <code>data-theme</code>	CSS <code>@media (prefers-color-scheme: dark)</code> tar over — fungerer alltid
<code>"dark"</code>	<code><html data-theme="dark"></code>	CSS <code>[data-theme="dark"]</code> -selektor overstyrer media-query
<code>"light"</code>	<code><html data-theme="light"></code>	CSS <code>[data-theme="light"]</code> -selektor låser lyst tema

Brave-fix: I system-modus brukes CSS `@media` som primærmekanisme — ikke JavaScript-events. Dette betyr at mørk modus fungerer selv om Brave ikke sender `change` -events til `matchMedia` . `visibilitychange` og `focus` -events brukes som fallback for å synkronisere `data-home-theme` -attributtet.

Toggle-syklusen



Sidestruktur — Saker og statistikk

Sider-siden (`/statistikk`) er delt i to tydelige seksjoner med egne overskrifter:

ØVERST — SAKER	NEDENFOR — STATISTIKK
----------------	-----------------------

Søkefelt, temafiltre og saksliste.

Henter fra `/eksport/saker`.

Klikk på sak → detaljer ved siden av.

Fuzzy-kategorisering i

`categorizationEngine.ts`.

KPI-kort (totalt, behandlet, til behandling, i kø).

Sesjonsfremdriftsbar.

Temafordeling og månedsgraf.

Alle tall fra samme `/eksport/saker`-kall.

Språkdeteksjon

Systemet auto-detekterer brukerens språk fra nettleseren og maskinen, uten å spørre brukeren eksplisitt:

```
// Sjekker navigator.languages i prioritetsrekkefølge
const browserSpråk = navigator.languages ?? [navigator.language]
const erNorsk = browserSpråk.some((l) =>
  l.toLowerCase().startsWith("nb") || // Bokmål
  l.toLowerCase().startsWith("nn") || // Nynorsk
  l.toLowerCase().startsWith("no") // Norsk (generelt)
)
return erNorsk ? "no" : "en"
```

Mobilnavigasjon — Hamburgermeny

På skjermer smalere enn 768px skjules den horisontale navbaren og erstattes av en hamburgermeny.

Navigasjonslenker: Forside, Saker, Stemmegivning, Representanter, Partiprogrammer, Om Sørblikket. Menyen lukkes automatisk ved:



Klikk på lenke

URL-ending via `useLocation()`



Escape-tasten

keydown event listener



Klikk på backdrop

Mørkt overlegg bak menyen

F — Driftsanvisning

Steg-for-steg guide for å drifte og oppdatere systemet.

Førstegangsoppsett

1. KLON OG INSTALLER

```
cd agderbenken/my-app && npm install
```

2. KONFIGURER MILJØVARIABLER

Lag `my-app/.env.local` med alle nøkler (se seksjon C).

3. KJØR EKSTRAKSJON

```
cd agderbenken && node scripts/extract-promises.mjs
```

Tar 15–45 minutter. Krever Anthropic-kreditter.

4. KJØR MATCHING

```
node scripts/match-promises.mjs
```

Tar 10–30 minutter avhengig av antall løfter.

5. START NETTSTEDET

```
cd my-app && npm run dev
```

Åpnes på <http://localhost:5173>

Løpende vedlikehold

Oppgave	Frekvens	Kommando
Oppdater voteringsstatuser	Ukentlig / etter stortingssesjon	<code>node scripts/update-status.mjs</code>
Kjør ny matching (nye saker)	Månedlig	<code>node scripts/match-promises.mjs</code>
Re-ekstraher ved nytt program	Kun ved partiprogramendringer	<code>node scripts/extract-promises.mjs</code>
Gjenprøv feilede partier	Etter feilmelding	<code>node scripts/retry-sp-r.mjs</code>

Vanlige årsaker til ingen treff

PROBLEM: 0% HOLDT PÅ PARTI-SIDEN

Årsak 1: Ekstraksjon ikke kjørt — databasen er tom.

Årsak 2: Matching ikke kjørt — ingen lofte_sak-koblinger.

Årsak 3: Anthropic-kreditter tomme under kjøring.

Løsning: Sjekk Supabase-dashbordet for rader i begge tabeller.

PROBLEM: SKRIPT KRASJER

Sjekk 1: Er `.env.local` på plass?

Sjekk 2: Er Anthropic-kreditter tilgjengelige?

Sjekk 3: Er Supabase-prosjektet aktivt?

Logg: Kjør med `> /tmp/out.log 2>&1` for full logg.

Kostnadsoversikt: Hvert API-kall til Claude (claude-opus-4-6) koster ca. \$15 per million input-tokens og \$75 per million output-tokens. Full ekstraksjon av alle 9 partier med 32 000 tokens per parti estimeres til \$5–15 per kjøring.